



**GETAMAZEDNOW AI**  
PRACTICAL AI. POSSIBLE FUTURES.  
A GETAMAZEDNOW COMPANY

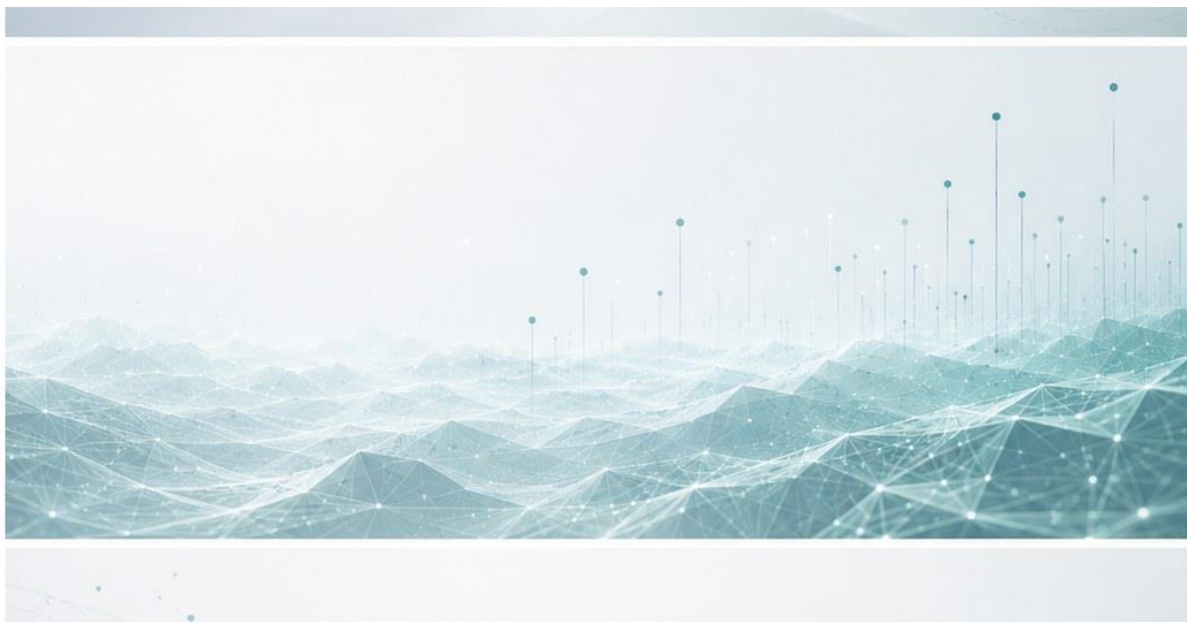
---

TRAINING GUIDE · MOCK / PORTFOLIO REFERENCE

# GenAI Project Observability

Observing agentic Gen AI workflows in production

A reference architecture and demo dashboard for observing agentic Gen AI workflows in production — built for architecture-board, risk-committee and Community of Practice review ahead of a real Datadog implementation.



**Version 1.2 FINAL DEMO DISTRIBUTION** · MOCK / SYNTHETIC DATA NOT CONNECTED TO ANY LIVE SYSTEM ·  
Prepared 2 July 2026

# GenAI Project Observability

## Table of Contents

|                                                                             |    |
|-----------------------------------------------------------------------------|----|
| 1. What this demo is .....                                                  | 3  |
| Purpose .....                                                               | 3  |
| Why conventional APM isn't enough .....                                     | 3  |
| The two source documents .....                                              | 4  |
| The demo scenario .....                                                     | 4  |
| Headline numbers from the current synthetic run .....                       | 5  |
| What "done" means for this v1 .....                                         | 5  |
| Related diagrams .....                                                      | 6  |
| 2. How to use the dashboard .....                                           | 7  |
| Getting it running .....                                                    | 7  |
| Navigation model .....                                                      | 8  |
| Suggested walkthrough order for a training session .....                    | 10 |
| Related diagrams .....                                                      | 10 |
| 3. Architecture choices, caveats and constraints .....                      | 11 |
| Core architectural choice: trace tree, not logs .....                       | 11 |
| Positioning: Datadog is a plane, not a policy .....                         | 12 |
| The single most important caveat: synthetic evaluation-harness series ..... | 12 |
| Other known gaps in this v1 (by design) .....                               | 12 |
| Caveats to carry into the real build (from the addendum, §10) .....         | 13 |
| Ownership model (accountability layers) .....                               | 13 |
| Reference standards this repository aligns to .....                         | 14 |
| Related diagrams .....                                                      | 14 |
| 4. Roadmap and next steps .....                                             | 16 |
| The 90-day path to production capability .....                              | 16 |
| What needs to happen to move from this demo to a real pilot .....           | 18 |
| Dependencies and sequencing risk .....                                      | 19 |
| Related diagrams .....                                                      | 19 |
| 5. Datadog APIs, monitors, connectors and implementation reference .....    | 20 |
| Instrumentation layer — which SDK/API to use .....                          | 21 |
| Proposed metric namespace (genai.*) .....                                   | 22 |

|                                                                                  |    |
|----------------------------------------------------------------------------------|----|
| Dashboards-as-code .....                                                         | 23 |
| Monitors .....                                                                   | 23 |
| SLOs .....                                                                       | 24 |
| What “connectors” means in this context .....                                    | 24 |
| Five-step migration checklist (mock → real) .....                                | 24 |
| 6. If Claude Code were to build the production infrastructure .....              | 25 |
| Operating principles for the agent .....                                         | 26 |
| Phase 0 — Preflight (human-only, before any agent work starts) .....             | 26 |
| Phase 1 — Instrumentation scaffold (agent-executable, sandboxed) .....           | 26 |
| Phase 2 — Ingestion wiring (agent-executable, requires scoped credentials) ..... | 27 |
| Phase 3 — Dashboards, monitors, SLOs (agent-executable) .....                    | 27 |
| Phase 4 — Governance and evaluation harness (largely NOT agent-executable) ..... | 27 |
| Phase 5 — Scale and reusability (Weeks 11–13 equivalent) .....                   | 28 |
| What an agent should never be handed unsupervised in this build .....            | 28 |
| Suggested sequencing summary .....                                               | 28 |
| Related diagrams .....                                                           | 29 |

## 1. What this demo is

### Purpose

This repository is a **portfolio reference architecture**: a working, fully-mocked demonstration of how to observe agentic Gen AI workflows in production, built to support architecture-board, risk-committee and Community of Practice conversations ahead of a real Datadog implementation. Nothing in it is connected to a live system — every number, trace and incident is generated by a Python script (`data/generator/`).

It exists to answer a question every enterprise adopting agentic Gen AI eventually has to answer: **what does “observability” mean for a system that can call tools, move money, retrieve from external sources, and occasionally hallucinate — and how do you actually build that on a platform like Datadog, not just describe it in a slide?**

### Why conventional APM isn’t enough

A Gen AI agent can return an HTTP 200 and still have failed. Conventional application monitoring (uptime, latency, error rate) misses the failure modes that matter most for agentic systems:

| Conventional APM sees      | Agentic Gen AI actually needs to catch                                                                                                                  |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| Request succeeded (200 OK) | Was the <i>business outcome</i> achieved, or did the agent silently produce a wrong or ungrounded answer?                                               |
| Latency within SLA         | Was cost within budget — did the workflow quietly burn 10x the expected tokens?                                                                         |
| No application errors      | Did a prompt-injection attempt bypass a guardrail and reach a money-moving tool?                                                                        |
| Server didn't crash —      | Did an agent loop fan out into 40 tool calls before anyone noticed?<br>Was a human review or approval step actually exercised where policy required it? |

This is the gap the two source documents address, and the gap this demo operationalises: cost, reliability, performance, agent behaviour, security and responsible-AI risk, all correlated by trace, in one observability model.

## The two source documents

Everything in this repository traces back to two documents (included in full under docs/source-documents/):

- **AI Observability Metrics Catalogue for Agentic LLM Workflows** (v1.0, 30 Jun 2026) — the *what to measure and govern* baseline. Roughly 150 metrics across 13 measurement dimensions (cost, performance, scale, reliability, agent behaviour, RAG/grounding, security, responsible AI, incidents, model quality/drift, data/privacy, change/release, and an executive view spanning all of them). Condensed in docs/metrics-catalogue.md.
- **AI Observability — Datadog Implementation Addendum** — the *how to implement it on Datadog* companion. Telemetry contract, seven-dashboard pack, SLOs, and a 90-day roadmap. Mapped in docs/datadog-mapping.md and docs/roadmap.md.

## The demo scenario

**Order Support & Returns Assistant** — an agentic retail customer-service workflow (order status, returns, refunds, loyalty credits, address changes) operating across three storefront brands, over a synthetic 30-day window (1 – 30 June 2026). This scenario was chosen deliberately because it exercises every dimension that matters for a governance-facing demo:

- **Cost at scale** — thousands of workflows a month, real token/tool cost economics.
- **Tool-driven side effects** — refunds move money; this is not a read-only chatbot.
- **A believable attack surface** — prompt injection against a refund\_issue tool is a realistic, high-stakes adversarial scenario, not a contrived one.

## Three seeded storylines

Three incidents are deliberately built into the synthetic data so the dashboards show a diagnosable signal, not noise — each one is a rehearsal for a real production scenario:

| Event                                | What happened                                                                                                                                                      | Why it's here                                                                                                                                                                    |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>REL-2026-0608 / REL-2026-0609</b> | A prompt version bump (v14 → v15) degraded groundedness and citation accuracy; caught by the (synthetic) evaluation harness and rolled back roughly 27 hours later | Demonstrates the Release & Evaluation dashboard's core job: catching a regression before it becomes a quality incident, and showing the eval-gate → rollback loop working        |
| <b>INC-2026-0611</b> (Sev2)          | A cluster of prompt-injection attempts targeting the <code>refund_issue</code> tool; the guardrail caught 92%, a handful bypassed                                  | Demonstrates the security dashboard's core job: catching an adversarial pattern and quantifying exactly what got through                                                         |
| <b>INC-2026-0621</b> (Sev1)          | A promo-driven traffic surge triggered provider rate-limiting, forcing retries and fallback-model routing, spiking latency and cost                                | Mirrors Datadog's own published finding that provider rate limits are a material share of production LLM failures — this is the “boring but common” incident, not a hypothetical |

## Headline numbers from the current synthetic run

| Metric                                                         | Value                                   |
|----------------------------------------------------------------|-----------------------------------------|
| Workflows (30 days)                                            | ~5,100                                  |
| Success rate                                                   | ~95%                                    |
| Cost per successful workflow                                   | ~\$0.025                                |
| Forecasted monthly AI run-rate                                 | ~\$120 (against a \$150 assumed budget) |
| Estimated monthly cost avoidance vs. human-handled equivalent* | ~\$31,400                               |
| Incidents                                                      | 2 (1 Sev1, 1 Sev2) + 1 release rollback |

\*Illustrative only — assumes a \$6.50 fully-loaded cost per human-handled contact, a modelling assumption for this demo, not a sourced benchmark. Regenerating the data will reproduce slightly different exact figures (the generator uses randomised-but-seeded distributions).

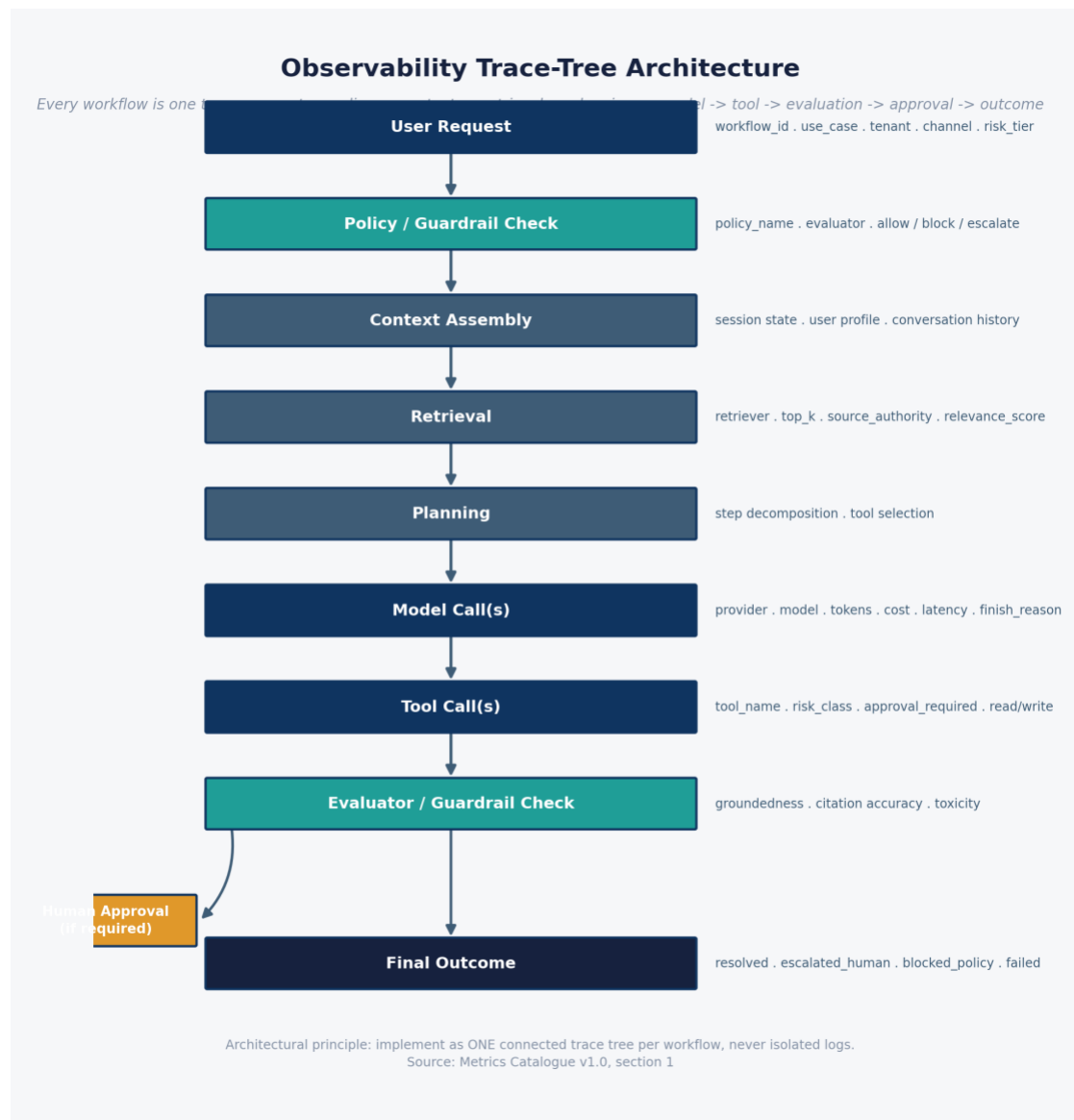
## What “done” means for this v1

All 7 dashboards from the Datadog addendum's pack are built and populated. The synthetic data pipeline mirrors, field for field, the shape of a real Datadog ingestion pipeline — see

02-dashboard-user-guide.md for how to read the dashboards, and 03-architecture-and-caveats.md for exactly what is and isn't modelled honestly in this v1.

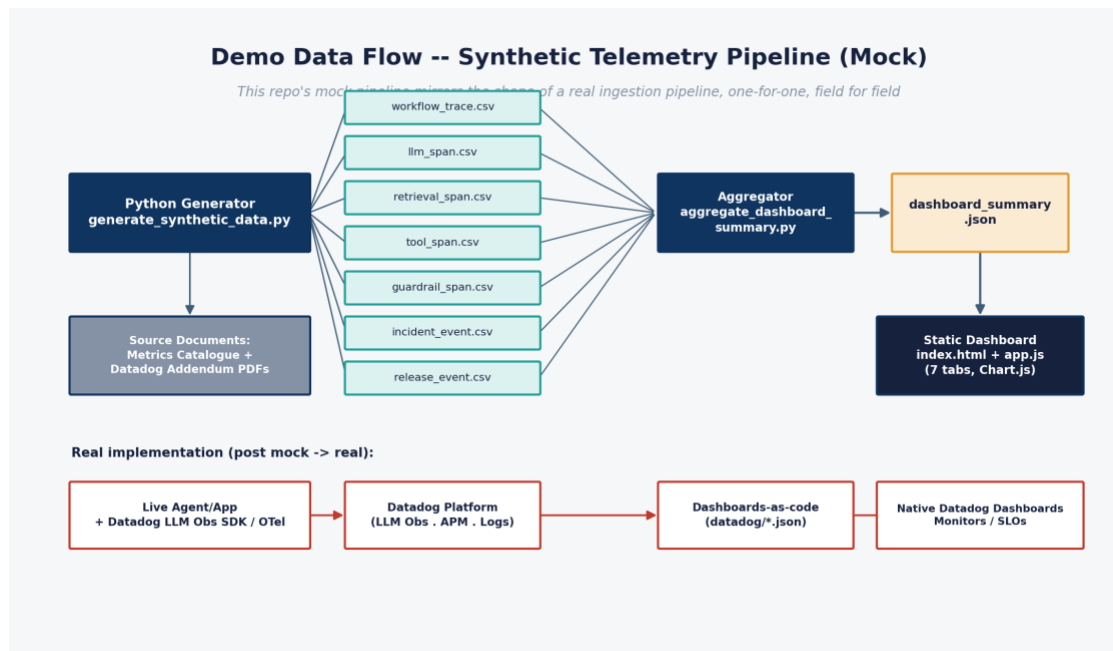
## Related diagrams

**Diagram — observability trace-tree architecture** (the principle behind every dashboard):



### Observability trace-tree architecture

**Diagram — demo data flow, synthetic telemetry pipeline** (how the synthetic data is generated and consumed):



*Demo data flow, synthetic telemetry pipeline*

## 2. How to use the dashboard

### Getting it running

The dashboard is a static HTML/CSS/JS app (Chart.js via CDN) — no build step, no server-side code.

```
cd genai-observability-demo/dashboard
python3 -m http.server 8000
# open http://localhost:8000
```

Browsers block `fetch()` of local JSON over `file://`, so it must be served (locally, or via GitHub Pages — `dashboard/` is fully static and Pages-ready). If you want to regenerate the underlying synthetic data first:

```
cd genai-observability-demo/data/generator
python3 generate_synthetic_data.py      # writes data/synthetic/raw/*.csv
python3 aggregate_dashboard_summary.py  # writes
dashboard/data/dashboard_summary.json
```

Every screen carries a **“MOCK / SYNTHETIC DATA”** pill in the header — keep that visible in any live demo or screen-share; it’s the honesty mechanism that makes this useful in a governance conversation rather than misleading in one.

## Navigation model

Seven tabs across the top, one per dashboard in the addendum's pack. All read from a single pre-aggregated file, `dashboard/data/dashboard_summary.json`, itself built from the raw trace-tree span data (`data/synthetic/raw/*.csv`) by the aggregator script. See the dashboard information architecture diagram below for how the seven map to audiences.

### Tab 1 — Executive AI Health

*Audience: C-suite, board, risk committee.*

- **KPI row:** workflow volume, success rate, cost per successful workflow, monthly run-rate, estimated cost avoidance vs. a human-handled equivalent, P95 latency, incident count.
- **Daily workflow volume by outcome** — resolved / failed / escalated / blocked, stacked by day.
- **Success rate (%) — daily trend** — watch for dips around the two seeded incidents.
- **Cost per successful workflow (USD) — daily trend** — the single number a CFO will ask about first.
- **Workflow volume by tenant (brand)** — adoption across the three storefront brands.
- **Cost composition — 30-day total by category** — LLM vs. tool vs. retrieval cost.
- **Incident log (Sev1/Sev2)** — the two seeded incidents, in a format a risk committee would expect to see.

*How to present it:* this is the tab for a 2-minute exec walkthrough — lead with cost avoidance and success rate, point at the incident log to show detection is working, and stop there unless asked to go deeper.

### Tab 2 — Engineering Operations

*Audience: platform engineering, SRE.*

- **Latency by layer (P95, ms)** — decomposes end-to-end latency into model / retrieval / tool time, so an on-call engineer knows where to look first.
- **P50/P95/P99 latency trend and error/retry/timeout rate.**
- **Provider rate-limit events — daily** — this is where INC-2026-0621 (the Sev1) shows up as a spike.
- **Agent loop events — daily** — repeated tool-call sequences; a leading indicator of runaway agent behaviour.
- **Representative trace waterfall** — a single illustrative trace, decomposed into guardrail → retrieval → model (with a rate-limited retry + provider fallback) → tool steps. This is the closest thing in the static demo to a real APM/LLM-observability trace view.



### Tab 3 — Security & Responsible AI

*Audience: cybersecurity, risk, responsible-AI governance.*

- **Prompt injection attempts — blocked vs. bypassed (daily)** — this is where INC-2026-0611 shows up: a visible cluster of attempts, with the bypass count exposed rather than buried.
- **Sensitive data egress & toxicity flags — daily.**
- **High-risk tool calls & policy approval bypasses — daily.**
- **Incident spotlight — INC-2026-0611** — a narrative callout box walking through what happened, detection, and mitigation.

*Why this tab matters most for a risk committee:* it's the one place a “successful” workflow (agent responded, no application error) and a “safe” workflow are shown as two different things.

### Tab 4 — Cost & Capacity

*Audience: FinOps, product owner.*

- **Token usage — input vs. output (daily).**
- **Cost by model (daily, USD).**
- **Provider quota utilisation (%)** against an assumed daily LLM-call quota — the promo-driven rate-limit incident (days 21–22) shows up here as a quota breach.
- **Cumulative cost vs. monthly budget** — a budget-burn line.
- **Fallback-model cost uplift (daily, USD)** — the direct dollar cost of routing to a more expensive fallback model during the rate-limit incident; this is the number that turns “we had an outage” into “the outage cost us \$X.”

### Tab 5 — Agent Behaviour

*Audience: AI/agent engineering, product.*

- **Average step count / tool calls per workflow — daily** — a proxy for agent efficiency and scope creep.
- **Tool calls by name (30-day total)** — which tools the agent actually uses.
- **Human escalation rate (%) — daily** — the automation boundary in practice.
- **Tool call success rate (%)** — used here as a *proxy* for tool-selection quality, since this demo doesn't model ground-truth “correct tool for intent” labelling. Call this out explicitly if presenting to an audience that will ask for a real accuracy number.

### Tab 6 — RAG & Grounding Quality

*Audience: data/ML engineering, responsible-AI risk.*

- **Retrieval hit rate (%) — daily.**
- **Groundedness & citation accuracy — daily average.**

- **Hallucination rate (%) and abstention rate (%) — daily.**
- **Average retrieved-source freshness (days) — daily.**

**Read the fine print here.** Groundedness, citation accuracy, hallucination rate and abstention rate are generated as a **synthetic evaluation-harness series** — independent of the raw retrieval/model spans — because that’s how a real implementation works too: evals run on a schedule against sampled traffic, scored by an LLM-as-judge or human reviewer, not computed inline on every request. See `03-architecture-and-caveats.md` for why this is a deliberate design choice, not a shortcut being passed off as something else.

## Tab 7 — Release & Evaluation

*Audience: release engineering, QA, MLOps.*

- **Regression pass rate & golden-set accuracy (%) — daily.**
- **Release / rollback log** — this is where the seeded prompt v14 → v15 regression and rollback (REL-2026-0608 / REL-2026-0609) appears as a concrete row, not an abstraction.

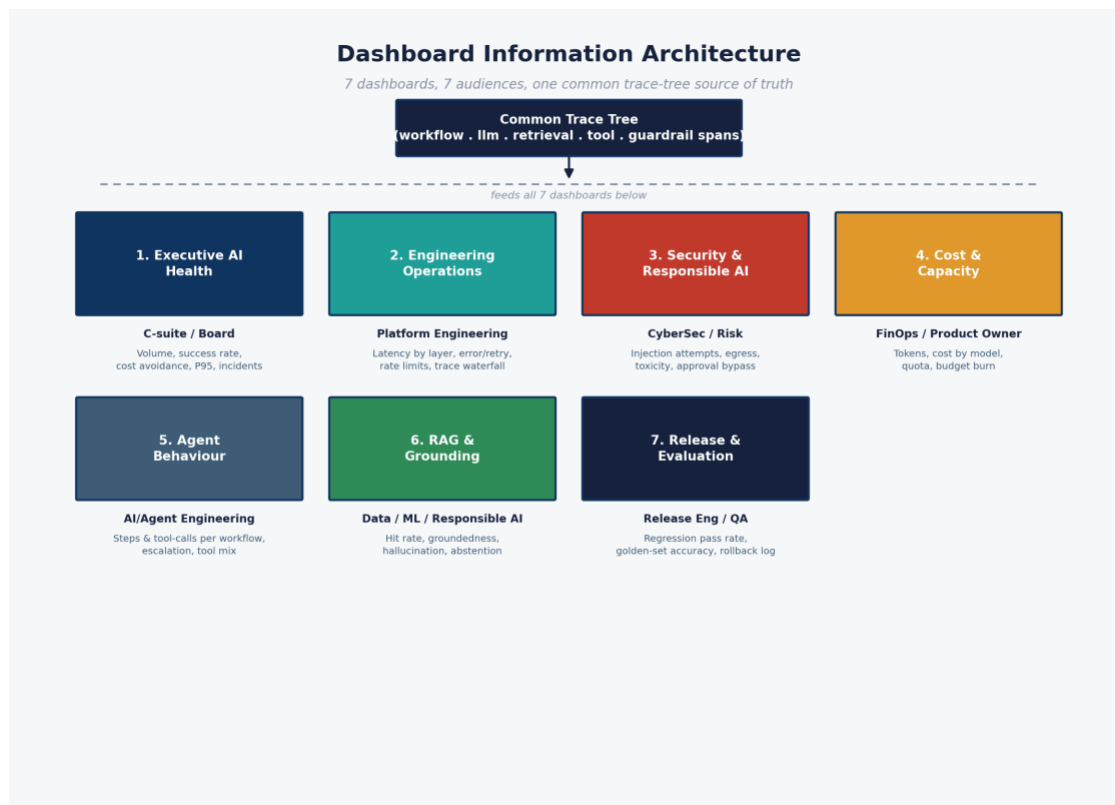
*Same caveat as tab 6 applies:* these scores come from the same synthetic evaluation-harness series, sampled on a schedule.

## Suggested walkthrough order for a training session

1. Start on **Executive AI Health** to set business context and headline numbers.
2. Jump to **Security & Responsible AI**, open the incident spotlight — this is usually the most attention-grabbing tab for a risk/governance audience.
3. Show **Engineering Operations**, specifically the trace waterfall, to ground the abstract “trace tree” concept in a concrete picture.
4. Show **Cost & Capacity** to connect the rate-limit incident to a dollar figure.
5. Close on **RAG & Grounding** or **Release & Evaluation**, explicitly flagging the synthetic-eval-harness caveat — this is where technical credibility is won or lost with a sophisticated audience.

## Related diagrams

**Diagram — dashboard information architecture** (7 dashboards, 7 audiences, one common trace-tree source of truth):



*Dashboard information architecture*

### 3. Architecture choices, caveats and constraints

Honesty about scope is the point of this repository. This page is the single place that consolidates every architectural choice, every known gap, and every caveat that should be carried into a board or risk-committee conversation — so nothing here is discovered for the first time in a harder room later.

#### Core architectural choice: trace tree, not logs

Per the metrics catalogue’s architectural principle, every workflow is instrumented as **one connected trace tree**: user request → policy checks → context assembly → retrieval → planning → model calls → tool calls → evaluator checks → human approvals → final outcome. See the trace-tree architecture diagram below.

This single choice is what makes cross-dimensional correlation possible — the same `workflow_id` that shows up as a cost line in the Executive tab is the same trace that shows a policy bypass in the Security tab. Isolated logs cannot do this; a trace tree can.

## Positioning: Datadog is a plane, not a policy

Per the addendum: **Datadog is the production observability/control plane, not the policy authority.** Policy ownership stays with architecture, security, data governance and responsible-AI risk functions. This demo mirrors that split deliberately — the synthetic guardrail spans record policy *decisions* (allow/block/escalate), they do not implement policy. That distinction matters in governance conversations: buying an observability tool does not discharge the accountability for setting the policy in the first place.

## The single most important caveat: synthetic evaluation-harness series

Groundedness, citation accuracy, hallucination rate, abstention rate, regression pass rate and golden-set accuracy are all present in the RAG and Release dashboards. They are generated as a **synthetic daily series**, independent of the raw workflow/LLM/tool/retrieval spans — this is deliberate, and it mirrors exactly how a real implementation has to work: evals run on a schedule against sampled traffic (an LLM-as-judge pipeline, golden sets, human review sampling), not inline on every request. Present this as an honest architectural point, not a hidden shortcut: **a real eval harness is still required to produce these numbers for real** (scoped into Weeks 7–10 of the roadmap).

## Other known gaps in this v1 (by design)

| Gap                                                                                  | Why it isn't modelled                                                                | Where it's tracked                                            |
|--------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|---------------------------------------------------------------|
| A real evaluation harness (LLM-as-judge, human review pipeline)                      | Needs actual model/human infrastructure a synthetic demo can't credibly fake         | Roadmap Weeks 7–10                                            |
| Multi-agent behaviour (handoffs, plan-revision count, critic/evaluator disagreement) | This scenario is deliberately single-agent                                           | Flagged in datadog/dashboards/agent-behaviour-and-agency.json |
| Bias/fairness flags, refusal quality, human override rate                            | No concept of a human editing or overriding an AI output exists in this scenario yet | docs/metrics-catalogue.md                                     |
| Ground-truth tool-selection accuracy                                                 | Tool-call success rate is used as a proxy instead                                    | Agent Behaviour tab, tab 5                                    |
| Timeout rate as a distinct signal                                                    | Only modelled via tool error_type=timeout, not as its own top-level metric           | Metrics catalogue, priority-metrics table                     |

None of these are oversights — they require capability (a real eval pipeline, multi-agent orchestration, human-review UX) that a synthetic-data demo cannot honestly fabricate. Calling them out explicitly is what separates a credible reference architecture from a demo that oversells itself.

## Caveats to carry into the real build (from the addendum, §10)

These are the six caveats the source addendum explicitly flags, and they should travel with this material into any real implementation conversation:

- **Don't overstate Datadog's market position.** Cite only what's evidenced (supported providers/frameworks) — not “used by every frontier lab.”
- **Observability ≠ governance.** Datadog operationalises controls; policy ownership stays with security/governance/risk/product, not the tooling vendor.
- **Prompt/output logging is a privacy decision, not a default.** Apply redaction, hashing, classification and retention classes by risk tier *before* logging real content — this needs to be decided before instrumenting live traffic, not after.
- **Standardise tags before scaling.** High-cardinality raw IDs as primary dimensions get expensive and noisy fast; agree the tag taxonomy in Weeks 0–2, not retrofit it later.
- **AI evaluations are probabilistic controls.** Pair LLM-as-judge scoring with human review, golden sets and deterministic policy checks for anything high-stakes — don't treat an eval score as a compliance control on its own.
- **Auto-instrumentation won't cover everything.** Proprietary orchestration or unsupported frameworks need manual spans/OpenTelemetry instrumentation — budget for this explicitly rather than assuming SDK coverage is complete.

## Ownership model (accountability layers)

Per the metrics catalogue, six functions each hold a distinct accountability slice. This should be assigned by name before any real workflow is instrumented — the addendum is explicit that observability tooling operationalises controls, it does not replace accountable risk ownership.

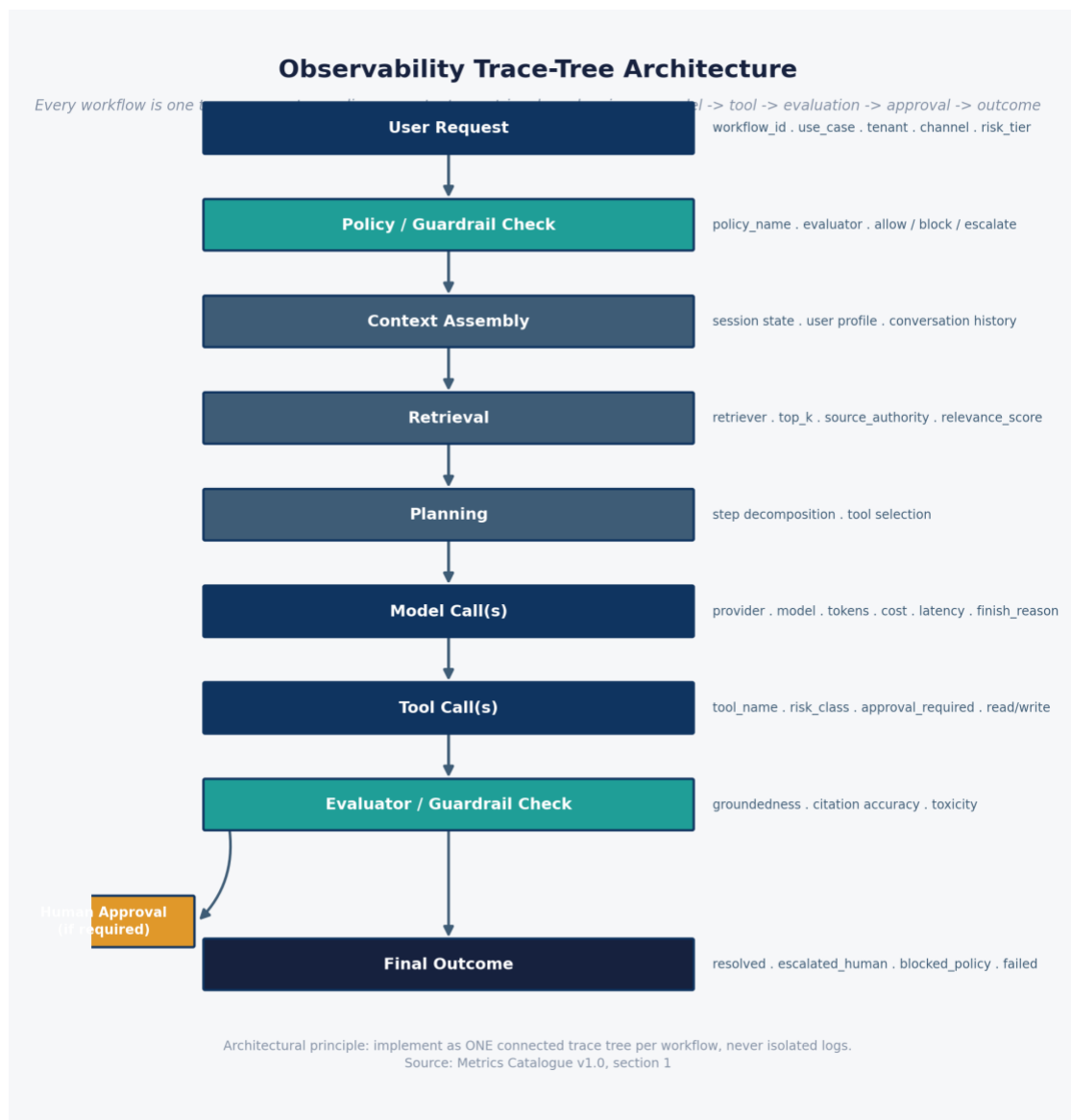
| Layer                 | Accountability                                                                                              |
|-----------------------|-------------------------------------------------------------------------------------------------------------|
| Platform Engineering  | Tracing, metrics, logging, latency, cost, quota, reliability, deployment observability                      |
| Cyber Security        | Prompt injection, data leakage, tool abuse, supply-chain risk, detection engineering, incident response     |
| Data Governance       | Source authority, classification, residency, retention, approved-source controls, data minimisation         |
| Responsible AI / Risk | Bias, fairness, explainability, human oversight, risk acceptance, model documentation, evaluation cadence   |
| Product Owner         | Business value, completion rate, adoption, user satisfaction, process impact                                |
| Architecture          | Patterns, guardrails, integration controls, model/tool governance, reference architecture, investment cases |

## Reference standards this repository aligns to

| Reference                                | Use                                                                                                                                 |
|------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| OpenTelemetry GenAI semantic conventions | Vendor-neutral traces, spans, metrics, model identity, token counts                                                                 |
| NIST AI Risk Management Framework 1.0    | Risk framing: valid/reliable, safe/secure/resilient, accountable/transparent, explainable, privacy-enhanced, fair                   |
| OWASP Top 10 for LLM Applications        | Security controls: prompt injection, insecure output handling, sensitive info disclosure, model DoS, supply chain, excessive agency |
| Datadog LLM/Agent Observability docs     | Implementation reference: traces, spans, evaluations, metrics, OTel ingestion                                                       |

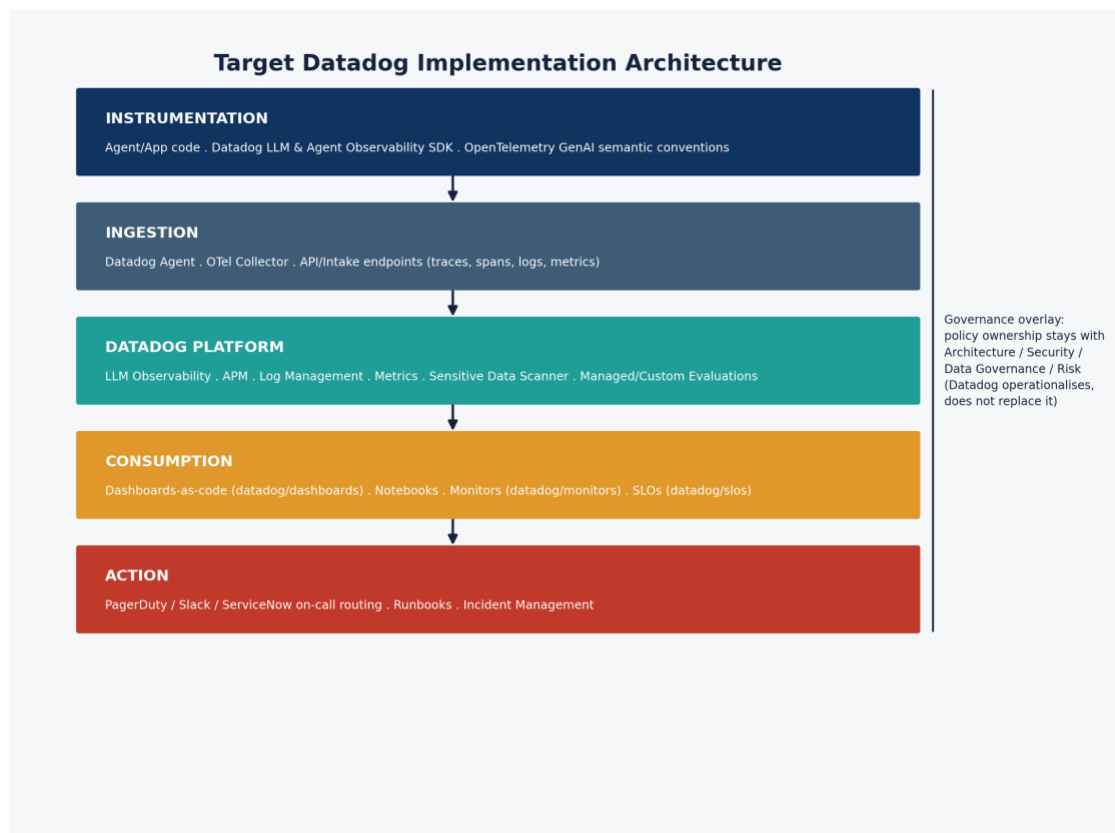
## Related diagrams

**Diagram — observability trace-tree architecture:**



*Observability trace-tree architecture*

**Diagram — target Datadog implementation architecture:**



*Target Datadog implementation architecture*

## 4. Roadmap and next steps

### The 90-day path to production capability

Source: *AI Observability — Datadog Implementation Addendum*, §5, annotated here against this repository's actual current status. See the timeline diagram below.





### 90-day roadmap timeline

| Horizon                               | Goal                                         | Core activities                                                                                                                                                | Exit criteria                                                                                                       | This repo's status                                                                                                                                                                               |
|---------------------------------------|----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Weeks 0-2<br/>—<br/>Foundation</b> | Establish standards and one observable pilot | Define AI telemetry contract, tag taxonomy, data retention policy, redaction policy, risk tiers, approved model/tool registry, first instrumented workflow     | One agentic workflow visible end-to-end with model spans, tool spans, token/cost/latency metrics and basic monitors | <b>Done in mock form.</b> Telemetry contract defined and matched to the addendum; one workflow (Order Support & Returns Assistant) fully instrumented — as synthetic data, not live traffic      |
| <b>Weeks 3-6<br/>— Operate</b>        | Move from traces to operational control      | Build dashboards for cost, latency, reliability, rate limits, agent loops, tool failures, safety evals; add SLOs, alert thresholds, runbooks, incident routing | Production support can detect, triage and assign AI workflow incidents via the platform, not manual log inspection  | <b>Done in mock form.</b> All 7 dashboards built; SLO/monitor definitions stubbed but not wired to a real alerting channel; three seeded storylines demonstrate the detection → triage narrative |

|                            |                                                     |                                                                                                                                                             |                                                                                                                           |                                                                                                                                                                                                                                                                     |
|----------------------------|-----------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Weeks 7–10 — Govern</b> | Operationalise responsible AI and security controls | Integrate Sensitive Data Scanner/DLP, prompt-injection signals, custom evaluations, human-review telemetry, audit evidence, high-risk tool approval metrics | High-risk workflows have policy-decision telemetry, audit evidence, approval/override tracking, responsible AI dashboards | but aren't live<br><b>Partially done.</b> Guardrail/policy-decision telemetry, approval-bypass tracking and a mocked evaluation-harness series are all modelled. Still not modelled: a real eval harness, bias/fairness flags, refusal quality, human override rate |
| <b>Weeks 11–13 — Scale</b> | Create reusable platform patterns                   | Publish instrumentation library/templates, service onboarding checklist, golden dashboards, SLO templates, cost-allocation model, release-gate criteria     | Second/third use cases onboard with materially less effort; governance and operations patterns are reusable               | <b>Not started.</b> This repo is a single-use-case reference; extracting it into a reusable instrumentation library is the next phase once a real workflow is live                                                                                                  |

## What needs to happen to move from this demo to a real pilot

This is the practical build-requirements list — what to line up before instrumenting the first real workflow.

### Organisational prerequisites

- Named owner assigned per accountability layer (Platform Engineering, Cyber Security, Data Governance, Responsible AI/Risk, Product Owner, Architecture) — see 03-architecture-and-caveats.md.
- Risk tier definitions agreed for the first pilot workflow (what makes a workflow “low”, “medium”, “high” risk — this drives SLO targets and approval requirements).
- A redaction/classification/retention policy signed off *before* any real prompt or output content is logged — this is a data governance and legal decision, not an engineering default.

- Executive sponsor identified who will own the cost-avoidance and ROI narrative (the Executive Health dashboard is built for this conversation).

## Technical prerequisites

- A Datadog account/org with LLM Observability and APM entitlements, and admin access to import dashboards-as-code, monitors and SLOs.
- Choice of instrumentation approach: Datadog’s LLM/Agent Observability SDK (where the framework is supported) vs. OpenTelemetry GenAI semantic conventions (for cross-vendor portability or unsupported frameworks) — see `05-datadog-implementation-reference.md`.
- An approved model/tool registry — which providers, models and tools are sanctioned for the pilot workflow, and their risk classifications.
- A tag taxonomy agreed and enforced *before* scaling past the pilot (per the addendum’s explicit caveat: high-cardinality raw IDs as primary dimensions get expensive and noisy fast).
- An on-call / alert-routing target (PagerDuty, Slack, ServiceNow) to wire the monitor and SLO definitions in `datadog/monitors/` and `datadog/slos/` to — they’re currently definitions only, not live.

## Capability prerequisites (the Weeks 7–10 gap)

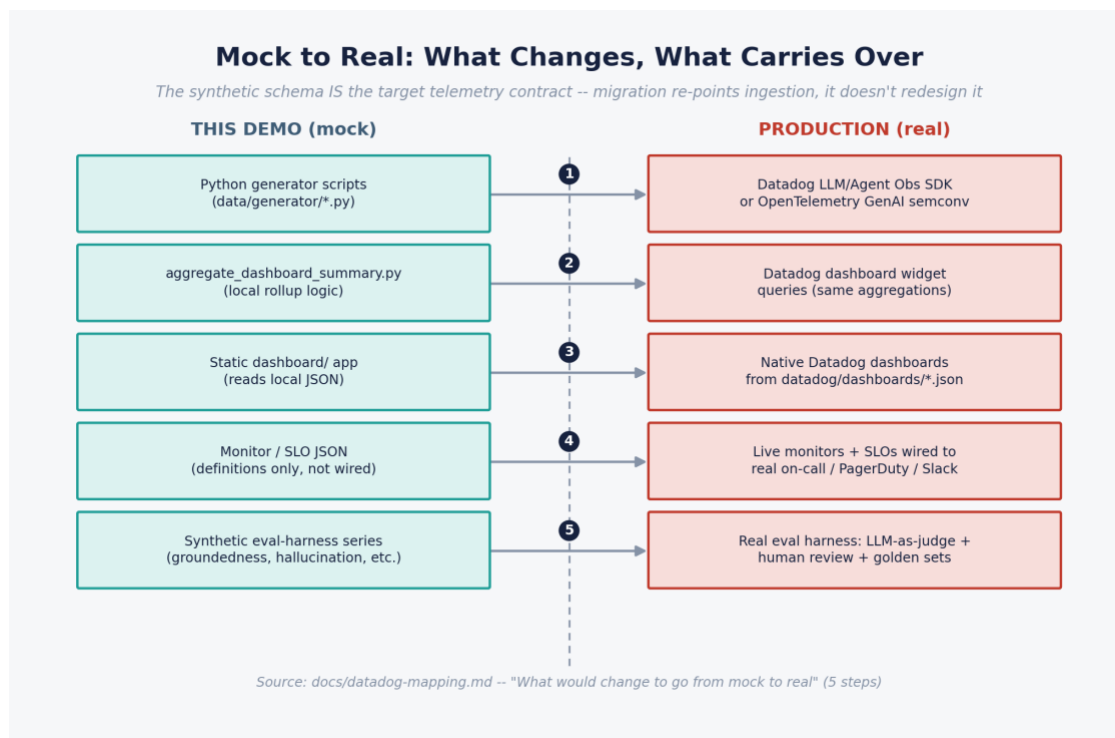
- A real evaluation harness: an LLM-as-judge pipeline, golden test sets, and a human-review sampling process — this is the single biggest capability gap between this demo and a defensible production system, because groundedness/citation-accuracy/hallucination/regression scores are currently synthetic.
- Sensitive Data Scanner / DLP integration for real prompt and output content.
- A human-override/human-edit concept in the product surface, if bias/fairness and override-rate metrics are to mean anything.

## Dependencies and sequencing risk

The roadmap is sequential for good reason: Weeks 3–6 dashboards are only trustworthy once Weeks 0–2’s telemetry contract and tag taxonomy are locked (retrofitting tags after scaling is expensive), and Weeks 7–10’s governance metrics are only meaningful once a real evaluation harness exists — building the Security/RAG dashboards on synthetic eval data indefinitely would quietly convert a demo into a false assurance. Treat the Weeks 7–10 phase as a hard gate before this pattern is presented as “production observability” to a risk committee, not just as “nice to have eventually.”

## Related diagrams

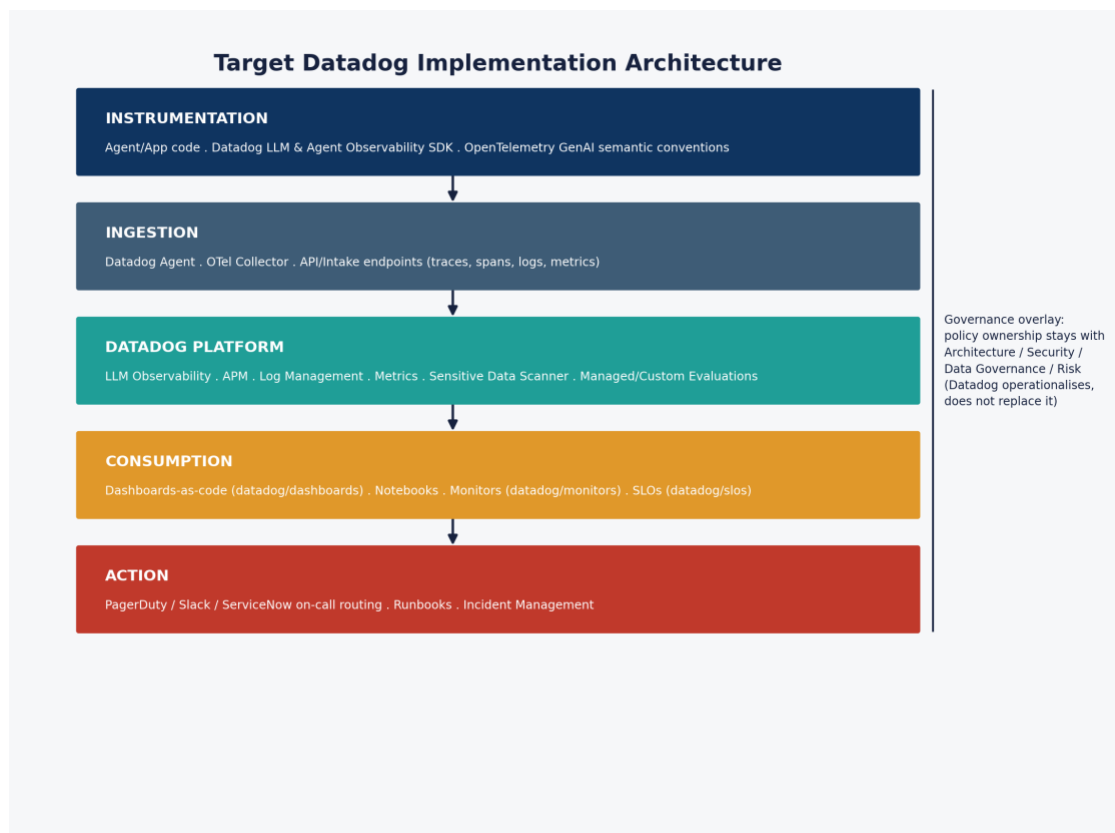
**Diagram — mock to real migration:**



*Mock to real migration*

## 5. Datadog APIs, monitors, connectors and implementation reference

This page is the technical bridge from `genai-observability-demo/datadog/` (the migration kit already in this repo) to a real Datadog tenant. Nothing described here is live — these are illustrative dashboards-as-code templates and monitor/SLO definitions, using a proposed metric namespace (`genai.*`) consistent with `../docs/datadog-mapping.md`. See the layered architecture diagram below.



### Target Datadog implementation architecture

## Instrumentation layer — which SDK/API to use

| Approach                                               | When to use it                                                                                        | Notes                                                                                                    |
|--------------------------------------------------------|-------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| <b>Datadog LLM Observability SDK</b> (docs)            | Supported frameworks/providers (OpenAI, Anthropic, Bedrock, Azure OpenAI, LangChain, LangGraph, etc.) | Auto-instruments spans, tokens, cost where supported; fastest path to production                         |
| <b>Datadog Agent Observability SDK</b>                 | Custom agent orchestration frameworks                                                                 | Extends LLM Observability to agent-specific spans (planning, tool calls, loops)                          |
| <b>OpenTelemetry GenAI semantic conventions</b> (spec) | Cross-vendor portability preferred, or proprietary/unsupported orchestration                          | Vendor-neutral; ingested by Datadog via the OTel Collector or Datadog Agent's OTLP intake                |
| <b>Manual spans / custom instrumentation</b>           | Anything auto-instrumentation doesn't cover                                                           | Addendum's explicit caveat: auto-instrumentation won't cover proprietary orchestration — budget for this |

The synthetic span taxonomy in this repo (workflow / llm / retrieval / tool / guardrail) mirrors the addendum's recommended span taxonomy (workflow, agent, llm, retrieval, tool, guardrail, human\_review, final\_response), so migrating the generator's field names into either SDK or OTel is a direct mapping exercise, not a redesign.

### Proposed metric namespace (genai.\*)

| Metric                                | Type         | Key tags                                             |
|---------------------------------------|--------------|------------------------------------------------------|
| genai.workflow.count                  | count        | outcome, use_case, tenant, channel, risk_tier        |
| genai.workflow.latency_ms             | distribution | use_case, tenant                                     |
| genai.workflow.cost_usd               | distribution | use_case, tenant, cost_category (llm tool retrieval) |
| genai.llm.call.count                  | count        | provider, model, status, finish_reason               |
| genai.llm.latency_ms                  | distribution | provider, model                                      |
| genai.llm.tokens                      | distribution | provider, model, token_type (input output)           |
| genai.tool.call.count                 | count        | tool_name, risk_class, status                        |
| genai.tool.latency_ms                 | distribution | tool_name                                            |
| genai.guardrail.decision.count        | count        | policy_name, allow_block_escalate, reason_code       |
| genai.incident.count                  | count        | severity, root_cause_category                        |
| genai.workflow.step_count             | distribution | use_case, tenant                                     |
| genai.retrieval.count                 | count        | retriever, hit (bool)                                |
| genai.retrieval.source_freshness_days | distribution | retriever                                            |
| genai.eval.groundedness_score         | gauge        | use_case — from evaluation harness, not raw traces   |
| genai.eval.citation_accuracy_score    | gauge        | use_case — from evaluation harness                   |
| genai.eval.hallucination_flag         | count        | use_case — from evaluation harness                   |
| genai.eval.abstention_flag            | count        | use_case — from evaluation harness                   |
| genai.eval.regression_pass_rate       | gauge        | artefact (prompt model) — from scheduled eval run    |
| genai.eval.golden_set_accuracy        | gauge        | artefact — from scheduled eval run                   |

|                                     |                      |                                                                            |
|-------------------------------------|----------------------|----------------------------------------------------------------------------|
| genai.release.event                 | count                | event_type<br>(release rollback),<br>artefact, from_version,<br>to_version |
| genai.release.active_prompt_version | gauge (tag<br>value) | use_case                                                                   |

Treat this list as a starting convention, not a fixed spec — LLM Observability auto-generates many equivalent metrics natively; the `genai.*` names above are for custom metrics where auto-instrumentation doesn't reach.

## Dashboards-as-code

`datadog/dashboards/` contains one JSON template per dashboard in the 7-dashboard pack, each flagged `"status": "implemented_in_demo"` and matching the 7 tabs in the static app:

| File                                         | Dashboard                                                                                                                                  |
|----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <code>ai-executive-health.json</code>        | AI Executive Health                                                                                                                        |
| <code>ai-engineering-operations.json</code>  | AI Engineering Operations                                                                                                                  |
| <code>ai-security-responsible-ai.json</code> | AI Security and Responsible AI                                                                                                             |
| <code>ai-cost-and-capacity.json</code>       | AI Cost and Capacity                                                                                                                       |
| <code>agent-behaviour-and-agency.json</code> | Agent Behaviour and Agency — carries a "caveat" field: single-agent scenario, multi-agent handoff metrics out of scope                     |
| <code>rag-and-grounding-quality.json</code>  | RAG and Grounding Quality — carries a caveat that groundedness/citation/hallucination scores are eval-harness output, not raw-span-derived |
| <code>ai-release-and-evaluation.json</code>  | AI Release and Evaluation                                                                                                                  |

**Import path:** Datadog UI → “Import dashboard JSON”, or programmatically via Terraform's `datadog_dashboard` resource, or the [Dashboards API](#).

## Monitors

`datadog/monitors/` holds five illustrative alert definitions, each tagged with its source clause in the addendum §8:

| Monitor                                 | Trigger                                                       | Severity routing        |
|-----------------------------------------|---------------------------------------------------------------|-------------------------|
| <code>workflow-success-rate.json</code> | Success rate < 98% (1h rolling) for low/medium-risk workflows | @pagerduty-genai-oncall |

|                                        |                                                                |                                                                  |
|----------------------------------------|----------------------------------------------------------------|------------------------------------------------------------------|
| rate-limit-error-rate.json             | Sustained provider rate-limit spike                            | Platform engineering on-call                                     |
| agent-loop-fanout.json                 | >25 loop/replanning events in 15 min (warning at 15)           | @pagerduty-genai-oncall — reliability <i>and</i> cost-risk event |
| high-risk-action-without-approval.json | Any high-risk guardrail bypass (zero-tolerance, threshold = 0) | @security-oncall @pagerduty-genai-oncall, tagged severity:sev1   |
| sensitive-data-egress.json             | Any confirmed sensitive-data egress event (zero-tolerance)     | Security/compliance on-call                                      |

**Import path:** Monitors API or datadog\_monitor Terraform resource. Each JSON's notify targets are placeholders (@pagerduty-genai-oncall, @security-oncall) — these need to be re-pointed to real on-call routing before going live.

## SLOs

datadog/slos/ defines SLO objects that reference the monitors above as time-slices:

| SLO                        | Target                     | Notes                                                                       |
|----------------------------|----------------------------|-----------------------------------------------------------------------------|
| workflow-success-rate.json | ≥98% (30d), warning at 99% | Business-outcome success (resolved + escalated_human) / total, not HTTP-200 |
| p95-latency.json           | Use-case specific          | Separate targets needed for interactive chat vs. batch agent workflows      |

## What “connectors” means in this context

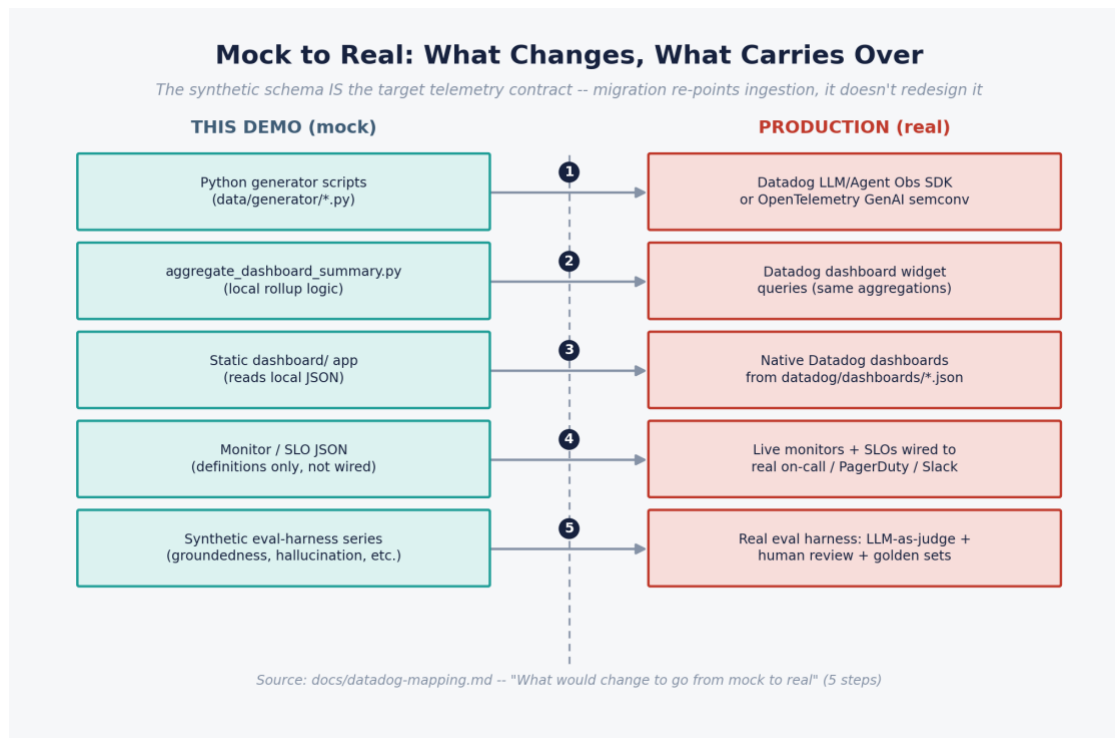
There is no single “Datadog connector” — the connection points are:

- **Ingestion:** Datadog Agent (host-based) or OpenTelemetry Collector (otlp receiver → Datadog exporter), or direct API/intake endpoints for traces, spans, logs and metrics.
- **Alerting egress:** Datadog's native integrations to PagerDuty, Slack, ServiceNow, Microsoft Teams, email and webhooks — configured once, referenced by name in every monitor's notify field.
- **Governance tooling:** Sensitive Data Scanner (built into the platform, not a separate connector) for real prompt/output content once actual data is being logged.
- **Evaluations:** Datadog Managed Evaluations (for supported patterns) or Custom Evaluations (bring-your-own LLM-as-judge/scoring pipeline) — this is the piece this demo cannot mock convincingly, per 03-architecture-and-caveats.md.

## Five-step migration checklist (mock → real)

See the mock-to-real migration diagram below.





### Mock to real migration

1. Replace `data/generator/*.py` with actual SDK/OTel instrumentation emitting the same field names into Datadog.
2. Replace `data/generator/aggregate_dashboard_summary.py` with real Datadog dashboard queries — the aggregation logic (daily P95, cost/successful-workflow, etc.) becomes the widget query definitions already drafted in `datadog/dashboards/`.
3. Replace the static dashboard/ app with native Datadog dashboards built from `datadog/dashboards/*.json` — or keep the static app as an executive-facing summary view fed by the Datadog API instead of the local JSON file.
4. Wire `datadog/monitors/*.json` and `datadog/slos/*.json` to real alert channels.
5. Stand up an actual evaluation harness (Datadog Managed/Custom Evaluations, an LLM-as-judge pipeline, golden sets, human review sampling) to produce the RAG/Grounding and Release/Evaluation tabs' numbers for real.

## 6. If Claude Code were to build the production infrastructure

This page is a **build plan and prompt spec**, written so it can be handed directly to Claude Code (or a comparable engineering agent) to execute the mock-to-real migration described in `04-roadmap-and-next-steps.md` and `05-datadog-implementation-reference.md`. It is scoped, phased, and gated with explicit human checkpoints — an agent should not be handed live API keys, money-movement tooling, or alert-routing configuration without a named human approving each gate.

## Operating principles for the agent

- **The synthetic schema is the target contract.** Do not redesign the telemetry schema; re-point it. Every field name in `data/synthetic/raw/*.csv` and `docs/datadog-mapping.md` should appear, unchanged, in the real instrumentation.
- **No live credentials in the agent's default context.** API keys, Datadog org access, PagerDuty/Slack tokens and any provider (OpenAI/Anthropic/Bedrock) credentials should be supplied only for the specific phase that needs them, via the human operator's own secrets management — never committed, never requested in chat.
- **Every phase ends with a human review gate,** not a merge-and-move-on. This mirrors the addendum's explicit point: observability tooling operationalises controls, it does not replace accountable ownership.
- **Additive, reversible changes.** Each phase should be a mergeable, revertable unit of work — no phase should require a later phase to be safe to ship.

## Phase 0 — Preflight (human-only, before any agent work starts)

Not delegable to an agent. Complete before Phase 1 begins:

- Named owner assigned per accountability layer (see `03-architecture-and-caveats.md`).
- Risk tier defined for the pilot workflow, and a redaction/retention policy signed off by data governance and legal.
- Datadog org provisioned with LLM Observability + APM entitlements; least-privilege service account created for the agent's CI pipeline (`dashboards/monitors/SLOs` import only — not full org admin).
- Approved model/tool registry for the pilot workflow.

## Phase 1 — Instrumentation scaffold (agent-executable, sandboxed)

**Goal:** replace `data/generator/generate_synthetic_data.py` with real instrumentation code, without touching production traffic yet.

Prompt-ready task for the agent: > Using `docs/datadog-mapping.md` as the field-level contract, scaffold Datadog LLM/Agent Observability SDK (or OpenTelemetry GenAI semantic-convention) instrumentation for the Order Support & Returns Assistant workflow. Emit spans for `workflow`, `llm`, `retrieval`, `tool` and `guardrail` matching the exact field names in `data/synthetic/raw/*.csv`. Do not connect to a live Datadog org yet — write to local OTLP export or console exporter for validation. Include unit tests asserting every required field from the telemetry contract table is present on each span type.

**Human gate:** review that span field names match the contract 1:1 before any real ingestion is enabled; confirm no prompt/output content is logged unredacted.

## Phase 2 — Ingestion wiring (agent-executable, requires scoped credentials)

**Goal:** point the Phase 1 instrumentation at a real (non-production) Datadog environment.

Prompt-ready task: > Wire the Phase 1 instrumentation to the Datadog Agent or OTel Collector, using the staging Datadog org credentials provided via [operator's secrets mechanism]. Validate that `genai.workflow.count`, `genai.llm.call.count`, `genai.tool.call.count` and `genai.guardrail.decision.count` (see the metric namespace table in `05-datadog-implementation-reference.md`) appear in Datadog's Metrics Explorer within 5 minutes of a test workflow run. Do not point at the production Datadog org.

**Human gate:** confirm staging data looks correct before promoting the same config to production; confirm redaction is actually being applied to any logged content, not just planned.

## Phase 3 — Dashboards, monitors, SLOs (agent-executable)

**Goal:** replace the local-JSON static dashboard with native Datadog dashboards, and make the monitor/SLO definitions live.

Prompt-ready task: > Import all 7 templates in `datadog/dashboards/*.json` into the staging Datadog org via the Dashboards API or `datadog_dashboard` Terraform resource, adapting metric names in each JSON to whatever Phase 1/2 instrumentation actually emits (auto-generated LLM Observability metrics vs. the `genai.*` custom convention). Import `datadog/monitors/*.json` and `datadog/slos/*.json` similarly. Leave all monitor notify targets pointed at a staging Slack channel, not the real on-call rotation.

**Human gate:** platform engineering validates dashboard queries against known-good staging traffic; only then is a second PR opened to re-point notify targets at the real on-call rotation (`@pagerduty-genai-oncall`, `@security-oncall`) — this re-pointing should be its own reviewed change, not bundled into the import.

## Phase 4 — Governance and evaluation harness (largely NOT agent-executable)

**Goal:** close the single biggest gap this repo is explicit about — replace the synthetic evaluation-harness series with a real one.

This phase is mostly infrastructure decisions and vendor/build choices a human team needs to make (Datadog Managed Evaluations vs. custom LLM-as-judge pipeline, golden-set curation, human-review sampling rate and reviewer assignment). An agent's role here is narrower and should be scoped explicitly:

Prompt-ready task (narrow, supporting role only): > Scaffold the plumbing to call [chosen evaluation service] on a scheduled basis against sampled production traffic, and emit

`genai.eval.groundedness_score`, `genai.eval.citation_accuracy_score`, `genai.eval.hallucination_flag`, `genai.eval.regression_pass_rate` and `genai.eval.golden_set_accuracy` (see metric namespace table) in the same shape the RAG and Release dashboards already expect. Do not select the evaluation methodology or curate golden sets — that decision belongs to Responsible AI/Risk and Data Governance.

**Human gate:** Responsible AI/Risk signs off on the evaluation methodology before its output is used to gate any release (i.e., before regression-pass-rate is wired into an actual release-blocking check).

## Phase 5 — Scale and reusability (Weeks 11–13 equivalent)

**Goal:** extract this single-use-case implementation into a reusable instrumentation library and onboarding pattern.

Prompt-ready task: > Extract the Phase 1–3 instrumentation, dashboard templates and monitor/SLO templates into a versioned internal package (or shared repo) with parameterised `use_case`, `tenant` and `risk_tier` values, and write an onboarding checklist a second team could follow without platform-engineering hand-holding. Do not change the underlying telemetry contract while doing this — this phase is about packaging, not schema redesign.

**Human gate:** a second, genuinely different use case attempts onboarding using only the checklist, with platform engineering observing but not intervening — if intervention is needed, the checklist has a gap.

## What an agent should never be handed unsupervised in this build

- Direct write access to a production Datadog org, production on-call rotation config, or production alert-routing — these should always be a reviewed PR, never a direct API call from an autonomous session.
- Money-movement tooling (refund/purchase tool credentials) in any context beyond the sandboxed/staging environment used for testing.
- The decision of what counts as an acceptable evaluation methodology or golden-set composition — this is a Responsible AI/Risk governance decision, not an engineering implementation detail.
- Redaction/retention policy definition — the agent implements the policy a human has already decided; it should not be asked to decide what's sensitive.

## Suggested sequencing summary

| Phase                        | Agent-executable?       | Maps to roadmap horizon |
|------------------------------|-------------------------|-------------------------|
| 0 — Preflight                | No (human-only)         | Weeks 0–2 prerequisite  |
| 1 — Instrumentation scaffold | Yes, sandboxed          | Weeks 0–2               |
| 2 — Ingestion wiring         | Yes, scoped credentials | Weeks 0–2               |

|                              |                               |             |
|------------------------------|-------------------------------|-------------|
| 3 — Dashboards/monitors/SLOs | Yes, staging-only until gated | Weeks 3–6   |
| 4 — Governance/eval harness  | Partially (plumbing only)     | Weeks 7–10  |
| 5 — Scale/reusability        | Yes, with acceptance test     | Weeks 11–13 |

## Related diagrams

**Diagram — phased sequencing maps to the mock-to-real migration and roadmap already introduced in this guide** (see 04-roadmap-and-next-steps.md and 05-datadog-implementation-reference.md for the full-size versions of the mock-to-real migration and Datadog architecture diagrams).